APIGUARD: A Machine Learning-Assisted Framework for Continuous API Security Assessment in DevSecOps Pipelines

Deliverable 2 — System Model and Description

Thornkasidej Fuangfoo (6622771234) Varit Mesri (6622771317)
Sirindhorn International Institute of Technology (SIIT), Thammasat University, Pathum Thani, Thailand
Adviser: Somchart Fugkeaw

***Abstract*—Rule-based linting is the only security check most OpenAPI specifications receive before deployment. A linter fires when a written rule matches, which leaves it unable to say which findings matter, unable to distinguish an endpoint returning a health-check string from one returning customer records, and silent about anything no rule covers. In a CI/CD pipeline this produces alert fatigue and blind spots at the same time. This document specifies the system model of APIGUARD, a framework that places a supervised classifier *after* the rule scanner rather than in place of it. The scanner keeps its authority over what rules cover; the model assigns every endpoint a risk score, ranks the findings, and explains each score with SHAP attributions, so that the pipeline gate acts on a risk score rather than on an alert count. We define the entities and their trust relationships, the notation and the gate decision rule, every runtime and offline component with its inputs, outputs and guarantees, the data flow of a single pipeline run, the threat model and its stated limits, and the deployment form. We also map each research question to the experiment that will answer it.**

***Index Terms*—API security, OpenAPI, DevSecOps, CI/CD, static analysis, XGBoost, SHAP, OWASP API Security Top 10, risk scoring.**

I. Introduction

Modern software is assembled out of HTTP APIs, and the OpenAPI specification is the first place where an API's security posture is written down: which endpoints require authentication, which scheme they use, what data comes back, and what limits apply. That makes the specification a cheap and early place to catch security design mistakes, long before a penetration test is run.

Today this checking is done by rules only. A linter such as Spectral with the OWASP ruleset fires when a written rule matches. It cannot say which of its findings actually matter, it produces the same alert whether an endpoint returns a health-check string or a customer's date of birth, and it says nothing about a problem nobody wrote a rule for. Inside a CI/CD pipeline this causes two failures at once: developers stop reading the alerts, and the gate lets dangerous designs through.

Scale makes the opportunity concrete. The APIstic dataset contains 1,275,568 valid OpenAPI specifications mined from GitHub, SwaggerHub, BigQuery and APIs.guru [1], and the curated APIs.guru directory alone holds 2,529 API descriptions covering 108,837 endpoints [8]. This is a large, machine-readable body of real-world API design, and it is almost untouched as training data for security.

APIGUARD places a supervised model *after* the rule scanner. The scanner keeps finding what rules find; the model gives every endpoint a risk score, ranks the findings, and explains each score with SHAP. The pipeline gate then acts on a risk score instead of a raw alert count. This document specifies the system model: the entities involved, the notation used, every component with its inputs and outputs, the data flow of a single pipeline run, the trust assumptions, and the way the system is deployed.

This deliverable describes the system for the registered topic "Continuous API Security Assessment in DevSecOps Pipelines." It replaces the Infrastructure-as-Code system presented in Deliverable 1, which was withdrawn because that topic had already been reserved by another group. The research design is unchanged; the target artefact is now the OpenAPI specification instead of Terraform code.

II. System Overview

APIGUARD is organised into three lanes. Separating them is a design decision rather than a drawing convenience: each lane runs at a different time, under different trust conditions, and at a different speed.

- The **runtime lane** is what ships. It runs on every commit inside the CI/CD pipeline, must finish in seconds, and is fully automatic.
- The **offline lane** builds the model. It runs rarely, on a workstation or build server, and may take hours.
- The **feedback lane** is how a human verdict re-enters the system. It is deliberately slow and never reaches the model directly.

Fig. 1 shows the three lanes and the two connections between them: the trained model artefact loaded by the runtime,

and the shared code path that guarantees training and runtime compute features identically.

III. Entities, Roles and Trust Assumptions

Six entities take part in the system. Table I states the role of each and how far it is trusted.

IV. Notation and Definitions

Table II fixes the notation used here and in every later deliverable.

For one specification S, the runtime lane computes the scored endpoint set

$$R(S) = \{\, (e,\ r(e),\ \phi(e)) \ :\ e \in E(S) \,\}, \tag{1}$$

and the gate decision is

$$G(S) = \begin{cases} 1, & \left|\{\, e \in E(S) \ :\ r(e) \geq \tau \,\}\right| > k, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

With the default policy $k = 0$ the gate fails if any single endpoint scores at or above τ. A larger k lets a team adopt the tool gradually on an existing API without blocking every build on the first day. The value of τ is not chosen by hand: it is swept on a validation set and reported as a precision/recall trade-off curve (Experiment E3).

Two properties the design must hold. First, $r(e)$ is defined for *every* endpoint, including endpoints on which no rule fired; for those, the scanner-derived features are simply zero. Second, for a fixed specification, ruleset version and model artefact the whole runtime lane is deterministic — nothing in it samples randomly.

V. Runtime Lane: Components S0–S3

A. S0 — Normaliser

Input: a specification file in YAML or JSON, either OpenAPI 3.x or Swagger 2.0. **Output:** a list of normalised endpoint records, one per (path, method) pair.

The normaliser resolves internal `$ref` pointers, expands the schema of every request body and response, and flattens the two specification versions into one internal form, so that no component after S0 needs to know which version the document came from. Path-level parameters are merged into each operation, and a global `security` block is propagated to operations that do not declare their own — recording, importantly, whether the operation *inherited* or *overrode* that requirement. An operation that explicitly overrides global security with an empty requirement is a different risk from one that simply inherits it, and a design that collapsed the two would lose that distinction before the model ever saw it.

Guarantees. Every endpoint of a parsable specification appears exactly once in the output. A specification that cannot be parsed is not silently skipped: it is written to an error log with the reason, and the corpus statistics report the parse-failure rate. Circular references are cut at a fixed depth so that expansion always terminates.

B. S1 — Rule Scanner

Input: the original specification file. **Output:** the set $A(e)$ of findings for each endpoint.

S1 runs Spectral with `@stoplight/spectral-owasp-ruleset` v2.x, which implements the OWASP API Security Top 10, 2023 edition [9], [12]. Spectral reports findings against JSON paths inside the document; S1 maps each finding down to the endpoint it belongs to, so that a document-level finding is attributed to every endpoint and an operation-level finding only to its own. The 42Crunch audit score may optionally be collected as an additional signal.

S1 output is used in three distinct ways, and keeping them apart matters for the evaluation: it is the **baseline** the model is compared against, it is the source of the **automatic labels** used for training, and it enters the model as **input features**. The scanner and ruleset versions are pinned and recorded in every report, because a ruleset upgrade changes both the baseline and the labels at the same time.

C. S2 — AI Verifier

S2 is the core of the contribution and has three sub-components.

S2a Feature builder. Turns one normalised endpoint record, plus its scanner findings, into a fixed-length numeric vector $x(e)$. The features fall into the six groups of Table III. The builder is a tested module with no dependency on the training pipeline, which is what allows the same code to be imported by both S2a and T3.

S2b Classifier. An XGBoost gradient-boosted tree ensemble [6], loaded from `model.json` at startup and applied to every endpoint. Gradient boosting is chosen because the input is a table of mixed numeric, binary and categorical features, because strong class imbalance is handled directly through `scale_pos_weight`, and because tree ensembles remain traceable back to individual features. Simpler baselines — majority class, logistic regression and random forest — are trained on the identical feature table for comparison.

S2c Explainer. SHAP [7] produces $\phi(e)$, attributing the score to individual features using Shapley values from cooperative game theory. Every finding APIGUARD reports carries the three to five features that pushed its score up. This is not decoration: it is how a developer checks whether the model reasoned about a security property or about something irrelevant, and it is how the team detects a model that has learned a shortcut.

D. S3 — Risk Gate

Input: $R(S)$, the scored endpoint set. **Output:** a report and a process exit code.

S3 applies the decision rule of (2) and emits the ranked findings in one of three formats: human-readable text for a terminal, JSON for further processing, and SARIF so that results appear as annotations in the code-review interface. The exit code is 0 on pass and 1 on fail, which is the only part of the system the CI runner needs to understand. The report always lists endpoints in descending risk order, and each entry

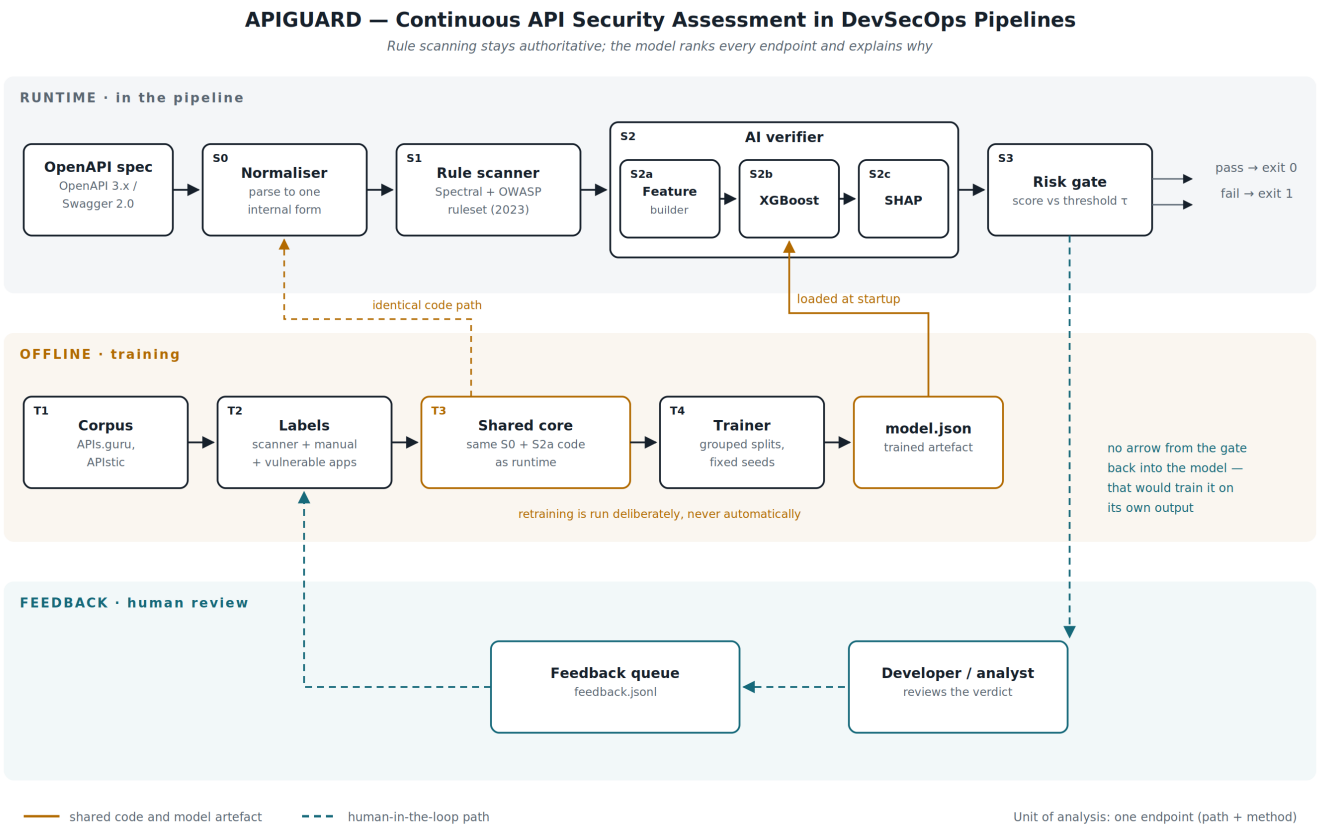


Fig. 1. APIGUARD system model. The runtime lane (S0–S3) runs inside the pipeline; the offline lane (T1–T4) produces `model.json`; the feedback lane returns human verdicts to the label store. Solid amber marks shared code and the model artefact; dashed teal marks the human-in-the-loop path. The unit of analysis is one endpoint.

TABLE I
ENTITIES, THEIR ROLES AND TRUST ASSUMPTIONS

Entity	Role in the system	Trust assumption
Developer	Writes or edits an OpenAPI specification and pushes it to the repository; reads the gate output and fixes what it reports.	Not malicious, but assumed to make design mistakes. Cannot change the gate threshold.
CI/CD runner	Checks out the repository, invokes the APIGUARD CLI, and acts on its exit code.	Trusted execution environment; its configuration is under version control.
APIGUARD CLI	The runtime lane, S0–S3. Reads a specification and produces a per-endpoint risk report and a pass/fail exit code.	The component under study. Must be deterministic for a fixed specification, ruleset version and model.
Rule scanner	Third-party linter (Spectral with the OWASP ruleset). Supplies findings used as a baseline and as model input.	Trusted but incomplete. Version is pinned, because a version change silently changes every label.
Security analyst	Reviews a sample of gate verdicts, marks each correct or incorrect, and owns the retraining decision.	Authoritative for labels. The only entity allowed to change what the model learns.
Offline trainer	Builds the corpus, assembles labels, trains the model and emits `model.json`.	Runs outside the pipeline. Never invoked automatically by a gate decision.

carries the score, the SHAP reasons, and any scanner findings that fired.

VI. Offline Lane: Components T1–T4

A. T1 — Corpus

Table IV lists the four sources. Each downloaded specification is stored with its source URL and a snapshot date, and is de-duplicated by normalised content hash before anything else happens. Public API specifications are heavily copied and templated between projects; near-duplicates left in the corpus would appear on both sides of a train/test split.

B. T2 — Label Store

Three label sources are kept separate and never merged into one undifferentiated pile.

- **L1 automatic.** Spectral OWASP findings mapped down to endpoints. Cheap and plentiful, and circular if used

TABLE II
NOTATION USED THROUGHOUT THE PROJECT

Symbol	Meaning
S	An OpenAPI 3.x or Swagger 2.0 specification document; the input to one pipeline run.
$e = (p, m)$	One *endpoint*: a path p combined with an HTTP method m. This is the unit of analysis.
$E(S)$	The set of all endpoints declared in S, produced by the normaliser S0.
$A(e)$	The scanner findings attributed to e, each carrying a rule identifier, a severity and an OWASP API Top 10 category.
$x(e) \in \mathbb{R}^d$	The feature vector of e, produced by S2a from the normalised specification and from $A(e)$.
$y(e) \in \{0, 1\}$	The label of e: 1 if the endpoint carries a security-relevant design issue, 0 otherwise.
$f : \mathbb{R}^d \to [0, 1]$	The trained classifier (XGBoost), loaded from `model.json` at CLI startup.
$r(e)$	The risk score of e, defined as $r(e) = f(x(e))$.
$\phi(e)$	The SHAP attribution vector for e: the contribution of each feature to $r(e)$.
$\tau,\ k$	The gate threshold and the tolerated count of endpoints at or above it.
$G(S)$	The gate decision for the whole specification: 0 = pass, 1 = fail.

TABLE III
FEATURE GROUPS COMPUTED BY S2A FROM ONE NORMALISED ENDPOINT

Group	Contents	Why it should carry signal
A. Structural	Path depth; number of path parameters; whether a path parameter looks like an object identifier (`id`, `uuid`, `_id`); HTTP method; query and header parameter counts; request body present; response schema property count; declared status codes; whether 401/403 are declared; deprecated flag.	An object identifier in the path with no ownership check is the classic API1 Broken Object Level Authorization shape.
B. Security scheme	Whether the endpoint has a security requirement; inherited versus overridden; scheme type (apiKey, http-basic, http-bearer, oauth2, openIdConnect); scope count; whether the server URL uses `http` rather than `https`.	The direct expression of authentication and authorisation design (API2, API5).
C. Data sensitivity	Count of response property names matching a sensitive-term lexicon (password, token, secret, ssn, dob, phone, email, address, card, balance); maximum term weight; sensitive properties in response versus request; ratio of sensitive to total properties; the combination flag *no security requirement and sensitive response fields*.	What a rule cannot see: it separates a health-check endpoint from one returning customer data (API3).
D. Documentation hygiene	Description and summary length; example present; readability index; version segment in the path; pagination parameters present; rate-limit headers documented.	Poorly specified endpoints correlate with weak design; rate-limit documentation relates to API4.
E. Scanner-derived	`flagged_by_spectral`, `rule_count`, `severity_max`, `owasp_categories_hit`, optionally `42crunch_audit_score`. All zero when nothing fired.	Lets the model use the rules without depending on them. Kept separable so the ablation can remove the group entirely.
F. Context	Endpoint count of the parent specification; OpenAPI 3 versus Swagger 2; provider or domain category.	Places the endpoint inside its API rather than judging it in isolation.

TABLE IV
CORPORA AND THEIR ROLE IN THE PROJECT

Source	Size	Use
APIs.guru [8]	2,529 APIs, 108,837 endpoints	Primary corpus; curated, clean, version-stable.
APIstic [1]	1,275,568 specifications	Scale-up if the schedule allows; reference for structural metrics.
VAmPI [10]	1 API, vulnerable *and* secure mode	Ground truth; the same API in two configurations gives matched positive and negative examples.
OWASP crAPI [11]	1 API, documented challenges	Second ground-truth target, independent of VAmPI.

alone — a model trained only on L1 can at best reproduce the scanner.

- **L2 manual.** A stratified sample of about 300 endpoints labelled independently by both team members, with disagreements resolved together and inter-rater agreement reported as Cohen's κ. Small, but this is the label set that makes the false-positive claim credible.
- **L3 ground truth.** VAmPI and crAPI endpoints with known, documented vulnerabilities.

C. T3 — Shared Core

T3 is not a separate implementation. It is the S0 normaliser and the S2a feature builder *imported* by the training pipeline, not copied into it. This is a deliberate constraint: if training and runtime computed features from two code paths, any drift between them would appear as an unexplained drop in live accuracy that no experiment could diagnose. The amber path in Fig. 1 marks this shared code.

D. T4 — Trainer

T4 takes the labelled feature table and produces `model.json`. Four rules govern it.

1) **Split by provider or repository, never at random.** A random split leaks near-identical endpoints across the boundary and produces impressive but meaningless scores. Grouped 5-fold cross-validation is used throughout.
2) **Report PR-AUC, F1 and MCC, not accuracy.** Insecure endpoints are a small minority; a model that

predicted "secure" for everything would score high on accuracy alone.

3) **Fix every random seed and pin every dependency version.** The scanner version, ruleset version, corpus snapshot date and seed are written into the model artefact's metadata.
4) **Emit an artefact, not a live service.** `model.json` is a file under version control, so any reported result can be reproduced with the exact model that produced it.

VII. Feedback Lane

When a developer or analyst disagrees with a verdict, the decision is appended to `feedback.jsonl` as one record per endpoint: the endpoint identity, the specification hash, the score it received, the human verdict, the reviewer and a timestamp. The queue is an input to T2 at the *next deliberate retrain*, run by a person who has looked at what is in the queue.

There is deliberately **no arrow from S3 back into S2b**. A model that consumes its own accepted predictions as training data reinforces its own mistakes and drifts away from the ground truth, with the drift hidden by the model's own agreement with itself. Human verdicts are therefore treated as new labels, not as confirmation.

VIII. End-to-End Data Flow

One pipeline run, in order:

1) A developer pushes a commit that touches `openapi.yaml`.
2) The CI runner checks out the repository and invokes `apiguard scan openapi.yaml --threshold 0.7`.
3) **S0** parses the file, resolves references and emits $E(S)$ — say 48 endpoint records.
4) **S1** runs Spectral with the pinned OWASP ruleset and attributes each finding to an endpoint, giving $A(e)$. Suppose 11 endpoints are flagged.
5) **S2a** builds $x(e)$ for all 48 endpoints, not only the 11. For the other 37, group E features are zero.
6) **S2b** loads `model.json` once and scores all 48, producing $r(e)$.
7) **S2c** computes $\phi(e)$ and keeps the top contributing features per endpoint.
8) **S3** ranks by $r(e)$, applies τ and k, writes the report and exits 0 or 1. An unflagged endpoint may appear at the top of the ranking — for example `GET /users/{id}` with no security requirement and a response containing `email` and `phone`.
9) The runner fails the build on exit code 1 and surfaces the SARIF annotations in the pull request.
10) The developer either fixes the specification or marks the finding as a false positive, which appends a record to `feedback.jsonl`.
11) At the next retrain, an analyst reviews the queue, T2 updates the labels, T4 produces a new `model.json`, and the new artefact is committed.

IX. Interfaces and Data Contracts

Table V states what each component accepts, what it returns, and the contract it must satisfy. These contracts are what allow the components to be unit-tested in isolation.

X. Assumptions and Threat Model

What is trusted. The CI runner and the repository are trusted. The specification file is trusted as an *input*, not as a *claim*: APIGUARD reads what it says but does not assume it is true of the running service.

Who the adversary is. The adversary is an external attacker interacting with the deployed API through its public interface, looking for endpoints that expose data without adequate authentication or authorisation, that leak sensitive fields, or that lack rate limiting. APIGUARD does not model an insider who tampers with the pipeline configuration, and it is not a defence against a developer who deliberately weakens the specification and lowers τ at the same time.

Assumptions. (i) The specification is a reasonably faithful description of the implemented API; where it is not, APIGUARD's finding is still useful as a documentation defect. (ii) Endpoints can be judged largely independently of one another; cross-endpoint attack chains are out of scope. (iii) The corpus of public specifications is representative enough of real-world API design to train on — an assumption tested directly by Experiment E4.

The central limitation, stated honestly. A specification can be secure on paper while the implementation behind it is not. APIGUARD measures *design-level* risk. It is not a substitute for dynamic testing and is not claimed to be. Experiment E4 measures how far design-level scoring carries into two deliberately vulnerable applications, and that measurement — whatever it shows — is reported as part of the result.

Out of scope. Source-code analysis of API implementations; GraphQL, gRPC and SOAP; live traffic analysis and WAF-style blocking; and dynamic scanning at corpus scale. OWASP ZAP and Schemathesis are used only in Experiment E4, against two applications, and never across the corpus.

XI. Design Justifications

Three decisions visible in Fig. 1 will be challenged in review, and each has a reason.

1) The model sits after the scanner, not instead of it. Rules stay authoritative for what they cover, and the model adds ranking plus coverage of what rules miss. This is the same arrangement Flynn *et al.* used to prioritise alerts from multiple static analysis tools with classification models [5]. The alternative — replacing the linter with a model — would trade a deterministic, auditable check for a probabilistic one, which is the wrong trade for a build gate.

2) S2 scores every endpoint, not only the flagged ones. Scanner output enters as features that are zero when nothing fired, so an endpoint no rule touched can still score high. Scoring only flagged endpoints would cap the model's recall at the scanner's recall and make the generalisation research question unanswerable.

TABLE V
COMPONENT INTERFACES AND THEIR CONTRACTS

Component	Input	Output	Contract
S0	Specification file (YAML/JSON)	Normalised endpoint records	Every endpoint appears once; parse failures are logged, never dropped silently.
S1	Specification file	Findings attributed per endpoint	The ruleset version is recorded in the output.
S2a	Endpoint record and its findings	Fixed-length feature vector	Same length and ordering for every endpoint; no null values.
S2b	Feature vector	Score in $[0, 1]$	Deterministic for a fixed model artefact.
S2c	Feature vector and model	SHAP attributions	Attributions sum to the score minus the base value.
S3	Scored endpoint set	Report and exit code	Exit code is 0 or 1 only; the report is always ranked by score.
T4	Labelled feature table	`model.json`	Carries seed, tool versions and corpus snapshot date in its metadata.

TABLE VI
RESEARCH QUESTIONS MAPPED TO EXPERIMENTS

Research question	Experiment	What is measured
RQ1 Feasibility. Does the model predict endpoint-level findings more accurately than the scanner alone?	E1 — provider-grouped 5-fold cross-validation on APIs.guru	PR-AUC, F1 and MCC against majority-class, scanner-severity and logistic-regression baselines.
RQ2 Generalisation. With one OWASP category removed from training, does the model still detect it?	E2 — rule hold-out (the headline experiment)	Recall on the held-out category's endpoints; above-chance recall means the model learned the design property, not the rule trigger.
RQ3 False positives. Does ranking reduce reading effort before the real issues?	E3 — analyst effort on the L2 hand-labelled sample	Precision@k and findings read to reach all true positives, versus scanner severity ordering; τ swept.
Supporting — external validity	E4 — VAmPI and crAPI	Whether known vulnerable endpoints rank above benign ones, and VAmPI secure mode below vulnerable mode.
Supporting — which features carry the signal	E5 — ablation and SHAP	Performance with each feature group removed, especially group E; global SHAP importance.

3) Feedback never reaches the model directly. Human verdicts enter through the label store at the next deliberate retrain, for the reason given above.

Position against related work. The related course topic "Automatic API Security Design Review for Microservices Using Threat Graph Analysis" performs threat modelling once, at design time, over a service graph. APIGUARD runs continuously inside the pipeline on every commit, at endpoint granularity, and produces a numeric gate decision with a per-finding explanation. The nearest published work, Cheh and Chen [2], builds a graph model of sensitive-field exposure in OpenAPI documents but evaluates it on a single API; recent surveys of RESTful API vulnerability detection [3] and of OpenAPI quality analysis [4] confirm that learned, explained risk scoring at corpus scale has not been attempted. That is the gap this project addresses.

XII. DEPLOYMENT VIEW

APIGUARD ships as a command-line tool that reads a file and returns an exit code, so it fits any CI system without a dedicated integration. A GitHub Actions job is the reference deployment:

```
- name: API security gate
  run: |
    apiguard scan api/openapi.yaml \
      --model models/model.json \
      --threshold 0.70 \
      --max-above 0 \
      --format sarif --out apiguard.sarif
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: apiguard.sarif
```

The model artefact is committed to the repository alongside the pipeline definition, so a build is reproducible from the checkout alone. Target runtime is a few seconds for a specification of a few hundred endpoints: S1 dominates, since S2b is a single batched prediction over a feature table. The gate can be run in report-only mode by setting a large k, which is the recommended way to introduce it to an existing API before it is allowed to block a build.

XIII. HOW THE SYSTEM MODEL WILL BE VALIDATED

The system model above defines what the next deliverable must measure. Table VI maps each research question to the experiment that answers it.

E2 is the experiment that answers the obvious criticism of the whole design — that a model trained on scanner labels has only learned the scanner. It is protected in the schedule ahead of every optional component.

XIV. CONCLUSION

This deliverable has specified the APIGUARD system model: three lanes separated by when they run and how far they are trusted, a runtime path of four components from specification to CI exit code, an offline path that produces the model artefact those components load, and a feedback path that returns human verdicts to the label store without ever closing a loop into the model. The notation and the gate rule in (1) and (2) fix what the implementation must compute, and the interface contracts fix what each component must guarantee. The next deliverable turns this model into the

proposed methodology and algorithm: the feature extraction procedure in full, the training procedure with its grouped splits and imbalance handling, and the protocol for the rule hold-out experiment that the whole design rests on.

REFERENCES

[1] S. Serbout and C. Pautasso, "APIstic: A large collection of OpenAPI metrics," in *Proc. 21st Int. Conf. Mining Software Repositories (MSR)*, Lisbon, Portugal, 2024, pp. 265–277, doi: 10.1145/3643991.3644932.

[2] C. Cheh and B. Chen, "Analyzing OpenAPI specifications for security design issues," in *Proc. 2021 IEEE Secure Development Conf. (SecDev)*, 2021, pp. 15–22, doi: 10.1109/SecDev51306.2021.00019.

[3] F. Tanveer, F. Iradat, W. Iqbal, and A. Ahmad, "Towards secure APIs: A survey on RESTful API vulnerability detection," *Computers, Materials & Continua*, vol. 84, no. 3, pp. 4223–4257, 2025, doi: 10.32604/cmc.2025.067536.

[4] A. Decrop, M. Vandeloise, P. Heymans, and G. Perrouin, "OASQuali: Automated quality analysis of OpenAPI specifications," in *Proc. Int. Conf. Web Engineering (ICWE)*, 2026, pp. 94–108, doi: 10.1007/978-3-032-29372-5_7.

[5] L. Flynn, W. Snavely, D. Svoboda, N. VanHoudnos, R. Qin, J. Burns, D. Zubrow, R. Stoddard, and G. Marce-Santurio, "Prioritizing alerts from multiple static analysis tools, using classification models," in *Proc. 1st Int. Workshop Software Qualities and Their Dependencies (SQUADE)*, Gothenburg, Sweden, 2018, pp. 13–20, doi: 10.1145/3194095.3194100.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[8] APIs.guru, "OpenAPI directory." [Online]. Available: https://apis.guru/

[9] Stoplight, "spectral-owasp-ruleset." [Online]. Available: https://github.com/stoplightio/spectral-owasp-ruleset

[10] E. Gavrilov, "VAmPI — vulnerable REST API." [Online]. Available: https://github.com/erev0s/VAmPI

[11] OWASP, "crAPI — completely ridiculous API." [Online]. Available: https://github.com/OWASP/crAPI

[12] OWASP, "API security top 10 — 2023 edition." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/